Indiecator

A Baremetrics-style analytics platform for indie SaaS — case study

Role	Solo full-stack engineer (architecture, backend, frontend, infra)
Client	Self-built · Live SaaS product
Status	Live in production
Stack	Node.js · Express · PostgreSQL · Prisma · Next.js 16 · React 19 · TypeScript · Tailwind · Recharts · Supabase Auth · Stri
Live	indiecator.com

TL;DR

Indie SaaS founders need the same revenue analytics as VC-backed companies — MRR, ARR, churn, expansion, LTV, ARPU — but Baremetrics and ProfitWell start at prices that don't make sense at $2k MRR. Founders end up exporting Stripe CSVs into spreadsheets, which breaks down the moment you have proration, mid-cycle upgrades, trials, or multiple products.

Indiecator is a self-serve analytics platform: connect Stripe via OAuth, get a fully reconstructed historical view of your business in minutes, and keep it in sync via webhooks.

Why this is harder than it looks

The hard part of MRR analytics isn't drawing the chart — it's deciding what number to draw. Stripe's API gives you invoices and subscriptions, not movements. To show a founder *why* their MRR went from $9.4k to $9.7k last month, you have to classify every change:

- A new subscription at $50/mo → **+$50 New MRR**
- An existing customer upgrading from $50 to $80 → **+$30 Expansion**
- A downgrade from yearly to monthly → **Contraction** (and the math is non-obvious)
- A canceled subscription → **Churn**, but *when*? At cancellation, or at period end?
- A trial converting → **New MRR**, but only at conversion, not trial start
- A proration invoice for $4.27 → **Not a movement at all**, just an accounting artifact

Get any of these wrong and the dashboard lies. Worse, the lies are silent — the chart still looks plausible.

Architecture

The data model centers on a single insight: **don't store MRR, store movements.** Every change to a customer's subscription is recorded as an `MrrMovement` row (`NEW | EXPANSION | CONTRACTION |`

`CHURN | REACTIVATION`) with a signed dollar impact and a price reference. Daily snapshots are derived by walking the movement history.

```
User ■< Project ■< Integration ■< { Customers, Subscriptions, Invoices, Charges }
                                                ↓
                                        MrrMovement (event log)
                                                ↓
                                    DailySnapshot (per-integration, per-day)
                                                ↓
                                Aggregation at query time → API
```

This shape means historical MRR is reconstructable from first principles, not cached. If a webhook is missed, the daily sync re-derives the truth from invoices.

The three-layer sync pipeline

- **Backfill** (one-time per integration, 9 steps): products → prices → customers → subscriptions → charges → invoices → movements (from invoice line items, not Stripe events) → snapshots → integration metadata.
- **Webhooks** (real-time): `invoice.paid` creates NEW movements, `subscription.updated` creates expansion/contraction by diffing against stored prior MRR, `subscription.deleted` triggers a shared churn helper.
- **Daily sync cron** (00:05 UTC reconciliation): re-pulls last 48h of invoices, detects missed churns, fills NEW movements that webhooks dropped, recomputes snapshots.

Each layer uses the same primitives, so a webhook outage is a non-event — the cron heals it within 24 hours.

Engineering decisions worth talking about

Movements derived from invoice line items, not Stripe events

Stripe's `subscription.created` tells you a subscription exists, not what the customer actually paid — promo codes, taxes, prorations, and discounts all live in the invoice. Sourcing NEW movements from `invoice.paid` line items means MRR matches what hit the bank account.

Idempotency with intentionally different windows per movement type

`createMrrMovement` deduplicates writes — but with different rules per type. NEW is one-per-subscription (you can never have two 'first' payments). Expansion/contraction/churn use a 60-second window keyed on subscription + type + signed amount, which absorbs webhook retries and Stripe's occasional double-fires without rejecting legitimate same-day changes.

Trial handling without polluting MRR

Trials are an analytics minefield. A `trialing` subscription has a price but no revenue. Indiecator excludes `trialing` from `isActiveMrrStatus`, stores trial invoices with `calculatedMrr = 0`, and only emits a NEW movement at trial *conversion* — not trial start. Reverse this and your 'New MRR' chart shows phantom revenue that vanishes when trials end.

Proration poisoning

Mid-cycle upgrades generate small proration invoices. Early on, the backfill was using these as the 'previous invoice' reference for the next billing comparison, which made every subsequent month's diff look like a downgrade. Fix: backfill skips updating `previousInvoice` when an invoice is `subscription_update` with `calculatedMrr = 0`.

Churn detection that refuses to invent revenue

Naively, when a subscription cancels you record churn equal to the subscription's amount. But for a sub on a 100% off coupon, the *real* MRR was $0 — recording $X of churn would make a no-op cancellation look like a revenue event. The churn helper queries the customer's last movement with `calculatedMrr > 0` and only falls back to the subscription amount if prior movements prove real MRR existed.

Weighted churn across integrations

Aggregating churn rate across a portfolio looks trivial until you do it. A founder with two products — one tiny test integration at 80% churn, one production at 3% — does *not* have 41% churn. Indiecator weights by total MRR rather than averaging rates, the kind of bug that's invisible in code review and obvious in a spreadsheet.

Frontend

Next.js 16 App Router with a context-driven scope system. A single `useContextMetrics` hook reads the current scope (portfolio / project / integration / plan) and period, fetches the right endpoint, and feeds the same chart components — so every metric page in the product is built from the same data layer.

Recharts components support drill-downs: click a bar on the MRR movement chart and you get the underlying customer list for that day. Period selector, formula bar (showing the literal math behind the headline number), and a unified loading/empty/error state are shared across every metric page.

Scale of the work

- **15+ Prisma models**, 840-line schema covering Stripe Connect's full surface area
- **65 backend source files** spanning OAuth, webhook handling, sync orchestration, and an aggregation layer
- **149 frontend TS/TSX files**, all using a shared scope-aware data layer
- **Full Baremetrics-style metric suite** — MRR, ARR, churn (MRR/user/revenue), expansion, contraction, reactivation, ARPU, LTV, cashflow, fees
- **Three independent sync paths** (backfill, webhooks, daily cron) reconciling to the same source of truth

What this project demonstrates

The visible product is a dashboard. The actual product is a sync pipeline that has to be correct under partial Stripe outages, webhook retries, trial edge cases, and proration math — because the cost of a wrong number on a founder's MRR chart is 'they stop trusting the tool.' Most of the engineering work wasn't building features; it was making numbers that look right *be* right. That's the bar I held this project to, and it's the bar I bring to production work.